

Chapter -> 11 Abstract classes & Interfaces

What does Abstract (class) mean?
 → Abstract in English mean → Existing in thought or as an idea without concrete existence.

Abstract Method

A method that is declared without an implementation
`abstract void moveTo (double x, double y);`

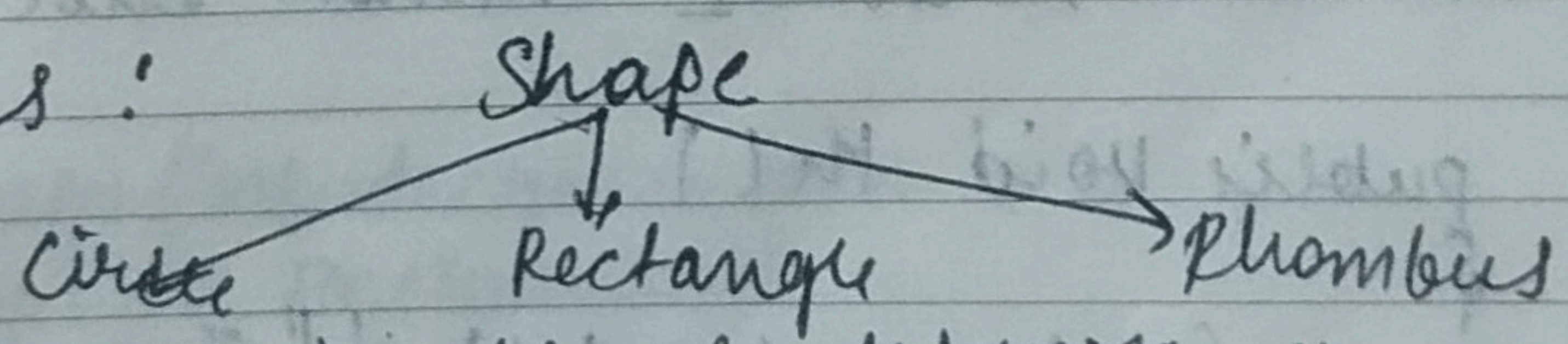
Abstract class

if a class includes abstract methods then the class itself must be declared abstract as in:

```
public abstract class PhoneModel {
    abstract void Switchoff();
    // more code
}
```

When an abstract class is subclassed, the subclass usually provides implementations for all of the methods in parent class. If it doesn't, it must be declared abstract.

Examples:



Note: It is possible to create reference of an abstract class
 It is not possible to create an object of an abstract class.

~~Interfaces~~

Example

Abstract class Base

{

public Base ()

{

System.out.println ("I am an abstract constructor")

}

2

public void sayHello ()

{

System.out.println ("Hello, I am a Java");

}

4

abstract public void greet ();

}

class Derived1 extends Base

{

@Override

public void greet ()

{

System.out.println ("Good morning");

}

}

abstract class Derived2 extends Base

{

public void hi ()

{

System.out.println ("I am Hi");

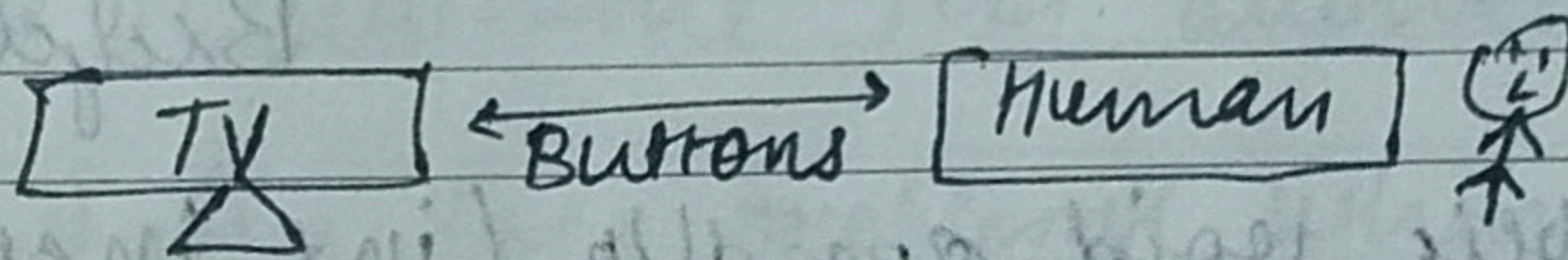
}

public class CHAPTER_11_ABSTRACT

```
{
    public static void main (String [] args)
    {
        Derived d1 = new Derived1 ();
        d1.greet ();
    }
}
```

Interfaces in Java

Interface in English is a point where two systems meet and interact.



In Java, Interface is a group of related methods with empty bodies.

Examples:

interface Bicycle

```
{
    int a = 45;
    void applyBrake (int decrement);
    void speedUp (int increment);
}
```



```

interface CreateCycle
{
    void applyHornK3g();
    void applyHornMhra();
}

class AyanCycle implements Bicycle, CreateCycle
{
    void blowHorn()
    {
        System.out.println("Pee Poo pee Poo Ppp");
    }

    public void applyBrake (int decrement)
    {
        System.out.println("Applying Brake in Bicycle");
    }

    public void speedUp (int increment)
    {
        System.out.println("Applying Speed up in Bicycle");
    }

    public void applyHornK3g()
    {
        System.out.println("Kabli Khushi Kabli Gum-----");
    }

    public void applyHornMhra()
    {
        System.out.println("Main Hoon Wal");
    }
}
  
```



```

public class CHAPTER-11 - INTERFACES
{
    public static void main (String args)
    {
        AvonCycle cycle = new AvonCycle ();
        cycle.applyBrake (5);
        cycle.applySpeedUp (10);
        cycle.applyHornK3g ();
        cycle.applyHornMha ();
        System.out.println (cycle.a);
    }
}

```

★ Why Multiple Inheritance not allowed in Java?

Multiple Inheritance face problems when there exists methods with same signature in the both super classes.

Due to such problems, Java does not support multiple inheritance directly but similar ~~inheritance~~ concept can be achieved using interfaces.

A class can implement multiple interfaces and extends a class at a same time.

SUN 09

Note: (1) Interface in Java is a bit like a class but with a significant difference.

② An interface can only have method signatures, fields and default methods.

③ The class implementing an interface needs to declare a method (not fields).

④ You can create a reference of ^{interfaces} ~~integers~~ but not the object.

⑤ Interfaces methods are public by default.

2

3

4

5

6

7

Default methods

In inheritance can have static and default methods. Default methods enable us to add new functionality to existing interfaces.

This feature was introduced in Java 8 to ensure backward compatibility while updating an interface. class implementing to interface need not implement the default methods.

Interfaces can also include private methods for default methods to use.

```
interface MyCamera {
```

```
    default void record4KCamera() {
```

```
        System.out.println("Recording in 4K - - -");
```

```
    }
```

```
}
```

```
    }
```

```
}
```

```
    }
```

```
}
```

```
    }
```

```
}
```

```
    }
```

```
}
```

```
    }
```

```
}
```


Inheritance in Interface

```
10 Interface SampleInterface {
```

```
    void meth1();
```

```
11    void meth2();
```

```
}
```

```
12 interface childSampleInterface extends implements SampleInterface
```

```
{
```

```
    void meth3();
```

```
    void meth4();
```

```
2 }
```

```
class MySampleClass implements childSampleInterface
```

```
3 {
```

```
    public void meth1 ()
```

```
    {
```

```
        System.out.println("meth 1");
```

```
    }
```

```
    public void meth2 ()
```

```
    {
```

```
        System.out.println("meth 2");
```

```
    }
```

```
    public void meth3 ()
```

```
    {
```

```
        System.out.println("meth 3");
```

```
    }
```

```
4    public void meth4 ()
```

```
    {
```

```
        System.out.println("meth 4");
```

```
    }
```

```
5 }
```


public class INHERITANCE-IN-INTERFACE

{
public static void main (String[] args)

{
MySampleClass obj = new MySampleClass ();

obj.meth1 ();

obj.meth2 ();

obj.meth3 ();

obj.meth4 ();
}

Polymorphism in interfaces